

Causal Seals: Decision Provenance for Governed Generation

M. H. EL hamiani — BATEN Technologies, Rabat, Morocco

Preprint — July 2026. Specification, schema, test vectors and reference implementation: github.com/causal-seal/causal-seal-spec · causalseal.org

ABSTRACT

Two families of standards address trust in AI-produced artifacts: content provenance (C2PA) proves *where an artifact comes from*, and integrity protocols prove *that a record was not altered*. Neither answers the question that regulators have now made statutory for high-risk AI systems: **why did the system produce this specific output?** We introduce **decision provenance**: a verifiable binding between a generated output and the **causal parameters** that governed its generation. We describe (i) *governed generation*, an architectural pattern that decouples deterministic control from stochastic text execution; (ii) the **Causal Seal**, an open, model-agnostic format that binds output, causal state and emission time under a recomputable SHA-256 fingerprint; and (iii) a zero-dependency reference implementation with computed test vectors, alongside a production deployment that has sealed every generated response since mid-2026. We map the format onto the record-keeping obligations of the EU AI Act (Articles 12 and 19), enforceable for high-risk systems as of 2 August 2026, and discuss what a seal does and does not prove. The specification, schema, vectors and implementation are released for community review.

1. Introduction

Large language models fail in a way that is more corrosive than error: they fail *unaccountably*. When a generation is wrong, no artifact explains which decision path produced it; when it is right, no artifact proves that the path was legitimate. The consequence is a permanent entitlement to doubt — and doubt is not repaired by better answers, only by proofs.

Regulation has moved faster than infrastructure. The EU AI Act requires high-risk AI systems to support automatic event recording ensuring "traceability of the functioning" of the system (Article 12) and the retention of those records (Article 19), with obligations enforceable from 2 August 2026. Comparable duties are emerging in United States state law and in procurement regimes anchored to the NIST AI Risk Management Framework. Yet the law prescribes the *obligation*, not the *technical form of a valid proof*. Content provenance standards protect media artifacts; audit-log protocols protect the integrity of records. The provenance of the **decision** — the configuration of control that made this output rather than another — has no open format.

This paper proposes one. Our contributions are deliberately modest in kind and, we believe, useful in consequence: a named architectural pattern, a small open format with a strict causality criterion, a reference implementation anyone can embed, and an honest account of the limits.

2. Related work

Alignment and fine-tuning (RLHF and successors) shape model behavior at training time; the resulting dispositions are neither inspectable per-response nor bound to outputs. **Guided decoding and guardrails** intervene at inference, but predominantly on the token distribution or on the final text, post-hoc; the intervention itself is rarely recorded in verifiable form. **Watermarking** detects machine origin without explaining machine reasoning. **C2PA** binds provenance metadata to media content — the right structure, aimed at a different object. **Hash-chained audit logs** and emerging cryptographic audit-trail protocols make records tamper-evident; they protect the diary, not the *why* — a faithful log of an unexplained decision is an unexplained decision, faithfully logged. **zkML** proves that a computation was executed correctly, which is orthogonal to proving which governing state ordered it. **IETF SCITT and RATS** provide transparency registration and environment attestation into which the present format slots naturally (§6).

To our knowledge, no open, model-agnostic format binds a generated output to the causal state of a *governance layer* under a publicly recomputable fingerprint.

3. The governed generation pattern

The pattern rests on one decoupling: **control of the reasoning trajectory is separated from linguistic execution**. A governance layer — implemented as deterministic state transitions over an explicit state (pure functions: same inputs, same outputs, no random draw in the control path) — computes, before and during generation, the configuration that will govern the response: which expert or route answers, under which regime and constraints, seeing which context. The language model executes text *under* that configuration; it is the executor of a trajectory, not the source of its own government.

This decoupling has a consequence that makes provenance meaningful: the governing configuration is **small, explicit, and frozen at emission time**. We call it the **causal state**, and we admit into it only parameters satisfying the **causality criterion**:

Had this parameter held a different value at generation time, the output could have been different.

Post-hoc descriptors of the output (token counts, readability scores) fail the criterion and are excluded by definition. The criterion is what separates decision provenance from decoration.

Two properties follow. First, **model- and temperature-independence**: whether the underlying model samples at temperature 0 or 1.2, the causal structure that ordered the task is fixed, transparent, auditable. Traceability does not require the model to be deterministic. Second, **conditional reproducibility**, which we state with care: where the full decoding path is controlled (own model, greedy decoding), identical causal states yield bit-identical text; on third-party sampled infrastructure this guarantee does not hold, and the contribution is causal traceability — *not* text reproduction. We regard stating this boundary as part of the contribution.

4. The Causal Seal format

The format is specified normatively elsewhere [SPEC]; we summarize its shape.

A seal is a small JSON object binding: a spec identifier, an emitter identifier, an ISO-8601 emission timestamp, the SHA-256 hash of the output (the output itself need not travel), the causal state, a reference to the emitter's **parameter dictionary**, and a fingerprint. The fingerprint is SHA-256 over the RFC 8785 (JCS) canonical serialization of the seal minus the fingerprint and optional signature — so two independent implementations serialize the same seal to the same bytes.

Parameter classes. Causal parameters are namespaced into six classes that answer an investigator's questions: *identity.** (*who* answered — route, expert, confidence), *engine.** (*with what* — the executing model), *field.** (*around what* — the semantic field state), *shaping.** (*how* — depth, fluidity, constraint filters, guard state), *context.** (*seeing what* — the memory actually visible to the model), and the timestamp (*when*).

The dictionary. Sealed values are auditable only if their meaning is public. Emitters must publish a dictionary giving, for each parameter: meaning, value domain, quantization. The dictionary discloses what a parameter *means* — not how the emitter *computes* it. This line is deliberate: it makes audit possible while leaving the engineering of good governance to competition. Mechanisms public in kind, calibration private in value.

Verification proceeds at three levels: **L1 integrity** (recompute the fingerprint — requires nothing but the seal), **L2 authenticity** (Ed25519 signature by the emitter's published key), **L3 causal audit** (cooperative: where the emitter's control path is deterministic, an auditor re-derives the same causal state from the same input). L3 is a property of emitters that the format makes *checkable* by giving auditors a precise target.

5. Reference implementation and deployment

The release accompanying this paper contains the specification, a JSON Schema, **computed test vectors** — a valid seal with its exact expected fingerprint, and a tampered seal that must fail — and a **reference implementation in a single dependency-free Python file** (emitter, Level-1 verifier, self-test), together with a browser-based public verifier computing entirely client-side. The self-test verifies the implementation against the vectors; any independent implementation can do the same.

Separately, a production conversational system operated by the author's organization has emitted a seal for every generated response since mid-2026, over a causal state of sixteen entries spanning the six classes. We report this not as evaluation but as an existence proof: the pattern and format run in production, at interactive latency, over third-party and local models alike.

6. Regulatory and infrastructure mapping

EU AI Act. Article 12 requires automatic recording of events ensuring traceability of functioning, identification of risk-relevant situations, and support for post-market monitoring; Article 19 requires providers to retain these logs. A seal per output, covering who/with-what/how/seeing-what/when, integrity-protected and optionally authenticated, addresses the technical core of these duties. We state the scope honestly: the format supplies the *capability* Article 12 presupposes; conformity of a deployed system remains its operator's obligation.

NIST AI RMF and US state regimes map similarly onto the Govern and Measure functions and onto documentation duties for consequential automated decisions.

Infrastructure. A seal is a natural payload for a SCITT signed statement (transparency, trusted time, non-equivocation); RATS-style attestation strengthens L2 into "which emitter, on which attested stack". Trusted timestamps may be added by RFC 3161 countersignature or append-only anchoring.

7. What a seal does not prove — limitations

A seal proves binding, not virtue. An emitter can seal meaningless parameters; the seal makes its claims fixed and auditable, not true. Trust is anchored the way transport security anchors it — the padlock proves the channel, not the merchant — through public dictionaries, L3 audits, attestation, and certification ecosystems. **Timestamps** are emitter-asserted absent countersignature. **Dictionary semantics** are not yet standardized across emitters; inter-emitter comparability of causal states is an open problem, as is the definition of domain profiles (which parameters a medical or financial deployment should minimally seal). **The pattern presupposes a governance layer**; retrofitting seals onto ungoverned pipelines yields thin causal states — arguably a feature, since the seal makes the thinness visible.

8. Conclusion

Between "trust the model" and "read the weights" there is a third road: govern the generation deterministically, and seal the government. The format proposed here is small on purpose — small enough to implement in an afternoon, strict enough that what it proves is worth proving. The specification, schema, vectors and reference implementation are open; we invite implementation, criticism, and the definition of domain profiles by the communities that will have to live with the results.

References

[SPEC] *The Causal Seal Format — Open Specification v1.0-draft*, 2026. github.com/causal-seal/causal-seal-spec

(Full bibliography in preparation for the journal version: EU AI Act (Regulation 2024/1689) Articles 12, 19; NIST AI RMF 1.0; C2PA specification; RFC 8785 (JCS); RFC 2119; RFC 3161; IETF SCITT and RATS working documents; representative literature on RLHF, guided decoding, watermarking, zkML.)

Preprint — July 2026 · Spec text CC BY 4.0, code MIT · Contact: contact@batenai.com · First conformant implementation: chat.batenai.ai